FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Jose Gabriel Valencia
Section:	An-43
Professor:	Ms. Tan
Date:	12/10/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

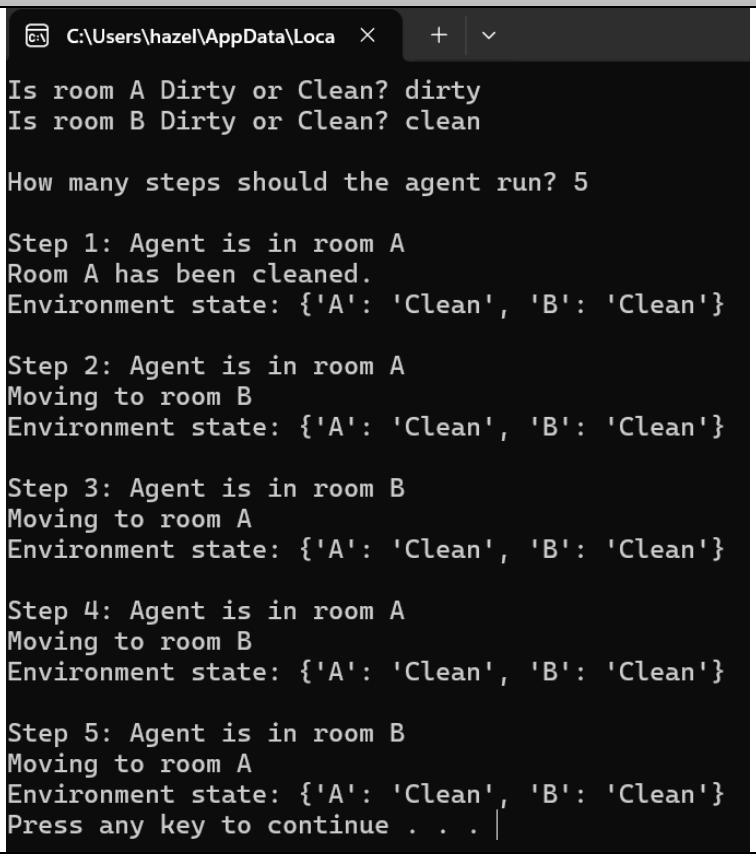
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

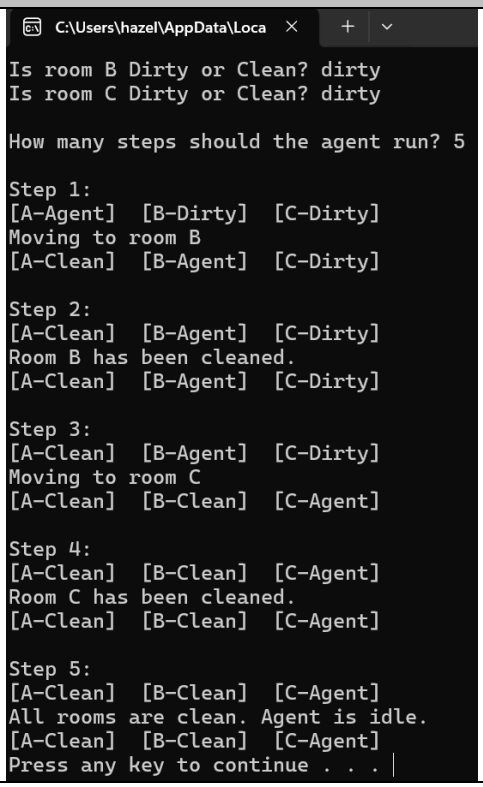
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
# Rule-Based Vacuum Cleaner Agent with Three Rooms


# Ask the user for a valid initial room state.
def get_room_state(room):
    while True:
        state = input(
```

```
            f"Enter the initial state of Room {room} "
            "(Dirty or Clean): "
        ).strip().capitalize()

        if state in ["Dirty", "Clean"]:
            return state

        print("Invalid input. Please enter Dirty or Clean.")


# Check whether a room is dirty.
def is_dirty(environment, room):
    return environment[room] == "Dirty"


# Clean the selected room.
def clean_room(environment, room):
    environment[room] = "Clean"


# Rule-based vacuum cleaner agent.
class RuleBasedAgent:

    def __init__(self):
        # The agent starts in Room A.
        self.location = "A"

    def move(self):
        # Move through the rooms in this order:
        # A -> B -> C -> A
        if self.location == "A":
            self.location = "B"
        elif self.location == "B":
            self.location = "C"
        else:
            self.location = "A"

    def perceive_and_act(self, environment):
        # Rule 1:
        # If the current room is dirty, clean it.
        if is_dirty(environment, self.location):
            room_cleaned = self.location
            clean_room(environment, room_cleaned)
```

```
            return f"Cleaned Room {room_cleaned}"

        # Rule 2:
        # If the current room is clean, move to the next room.
        previous_location = self.location
        self.move()

        return (
            f"Moved from Room {previous_location} "
            f"to Room {self.location}"
        )


# Set up the three-room environment.
environment = {
    "A": get_room_state("A"),
    "B": get_room_state("B"),
    "C": get_room_state("C")
}

# Create the agent.
agent = RuleBasedAgent()


# Ask for a valid number of simulation steps.
while True:
    try:
        number_of_steps = int(
            input("Enter the number of simulation steps: ")
        )

        if number_of_steps > 0:
            break

        print("Please enter a number greater than zero.")

    except ValueError:
        print("Please enter a valid whole number.")


# Display the initial state.
print("\nInitial Environment")
```

```
print(f"Agent Location: Room {agent.location}")
print(f"Room A: {environment['A']}")
print(f"Room B: {environment['B']}")
print(f"Room C: {environment['C']}")


# Run the simulation.
for step in range(1, number_of_steps + 1):
    action = agent.perceive_and_act(environment)

    print(f"\nStep {step}")
    print(f"Action: {action}")
    print(f"Current Location: Room {agent.location}")
    print(
        "Environment: "
        f"Room A = {environment['A']}, "
        f"Room B = {environment['B']}, "
        f"Room C = {environment['C']}"
    )


# Display the final result.
print("\nSimulation Complete")
print(f"Final Agent Location: Room {agent.location}")
print(
    "Final Environment: "
    f"Room A = {environment['A']}, "
    f"Room B = {environment['B']}, "
    f"Room C = {environment['C']}"
)
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. It follows the rules where if the room is dirty it will clean it and if its clean it moves to another room to check if its dirty or clean. This rule repeats until all the steps are used.

2. The agents check using its conditions if the room is label with the variable dirty or clean. If its dirty the agent cleans the room and stays there but if another step considers the room clean it moves to another room to inspect.

3. If both rooms are already clean the agent will not perform any cleaning action but instead moves to another room until all the steps are used.

4. Instead of writing separate rules for the room we can use an index of position for the agent where it browses through the rooms and upon reaching the final room it just returns to the first room like a loop until all the steps are used.